

Guión del diagrama de arquitectura

Notas del presentador (versión detallada) — Tienda Perritos · CI/CD + AWS EKS + HPA
(ISY1101)

Recorrido del diagrama fotos/arquitectura.svg de arriba hacia abajo. Cada bloque indica **qué señalar** y un texto sugerido para **decir**, con los comandos y valores reales del proyecto.

Apertura

Señalo: el título y la forma general del diagrama.

“Este diagrama resume toda la solución de Tienda Perritos y se lee de arriba hacia abajo: arriba el ciclo de desarrollo, y bajando vemos cómo un commit termina corriendo dentro de AWS y llegando al usuario. La región es us-east-1 y el clúster se llama tienda-eks. Todo lo que muestro está implementado y verificado.”

1 · Trigger del pipeline

Señalo: la franja morada (Desarrollador → GitHub Actions).

“Todo arranca cuando hago `git push` a la rama **main**. El workflow escucha `on: push: branches:[main]`, así que ese push dispara el job `build-and-deploy` sobre un runner `ubuntu-latest`.”

“El job declara `environment: production`, y recién ahí se inyectan los **7 secrets**: las tres credenciales de AWS más el `AWS_SESSION_TOKEN`, la región, el `EKS_CLUSTER_NAME`, el `EKS_NAMESPACE` y la `MYSQL_ROOT_PASSWORD`. Ningún secreto vive en el repositorio.”

2 · Autenticación contra AWS

Señalo: la flecha que baja de GitHub Actions hacia AWS.

“Los primeros pasos del pipeline son: `actions/checkout` baja el código; `configure-aws-credentials` usa las llaves y el **session token** —que es obligatorio porque las credenciales del Learner Lab son temporales y rotan cada sesión—; y `amazon-ecr-login` autentica Docker contra nuestro registro ECR.”

3 · Build y push a ECR

Señalo: la flecha naranja `docker push` y la caja de Amazon ECR.

“Aquí ocurre el **docker build y el docker push**. El pipeline fija `IMAGE_TAG=${GITHUB_SHA:0:7}`, o sea los 7 primeros caracteres del commit, y para cada componente hace `docker build` seguido de **docker push a ECR** con dos etiquetas: `:<sha7>` —inmutable y trazable al commit— y `:latest`.”

“El resultado son tres imágenes nuevas en los tres repositorios privados: `tienda-frontend`, `tienda-backend` y `tienda-db`, cifrados con AES-256. Este push a ECR es el que después consumen los nodos.”

4 · Conexión al clúster

Señalo: la flecha roja `kubectl` hacia el plano de control EKS.

“El pipeline instala `kubectl` y ejecuta `aws eks update-kubeconfig --name tienda-eks`, que genera el `kubeconfig` apuntando al endpoint del plano de control. Desde ahí ya puede mandar órdenes a la API.”

“Y aquí un concepto clave: un clúster EKS son dos mitades. Esta caja roja es el **plano de control** —API server, etcd, scheduler— y lo administra **AWS en su propia red**, por eso lo dibujé fuera de nuestra VPC. Nosotros no lo mantenemos: es el modelo de responsabilidad compartida.”

5 · Despliegue declarativo

Señalo: la caja VPC y, debajo, el namespace.

“Con el kubeconfig listo, el pipeline aplica los manifiestos **en orden**: primero `kubectl apply -f k8s/namespace.yaml` para el namespace tienda.”

“Luego crea el Secret **en caliente**: `kubectl create secret generic mysql-secret --from-literal=... con --dry-run=client -o yaml | kubectl apply -f -`, tomando la contraseña del secret de GitHub. La contraseña nunca se versiona.”

“Después, para db, backend y frontend repite el patrón: `kubectl apply` del deployment y el service, `kubectl set image` a la imagen recién pusheada por su tag de commit, y `kubectl rollout status` que espera a que el rollout termine sano antes de seguir.”

6 · Qué pasa en los nodos

Señalo: las dos cajas ‘Nodo EC2 (privado)’ y el borde de red (IGW, NAT, ELB).

“Esta franja azul son las subredes privadas en 2 zonas de disponibilidad. Dentro están nuestras 2 EC2 t3.medium con Amazon Linux 2023, sin IP pública. El **kubelet** de cada nodo recibe la orden, hace el **pull de la imagen desde ECR** —y como no tienen IP pública, esa descarga sale por NAT Gateway → Internet Gateway— y levanta los pods.”

“Importante distinguir: el **nodo es la máquina** (la EC2) y el **pod es la app** que corre encima. Tenemos 2 nodos y, en reposo, 5 pods. Lo único expuesto a Internet es el ELB de las subredes públicas; los nodos quedan protegidos.”

7 · El node group

Señalo: la caja ‘Escalado del node group’.

“Las 2 EC2 pertenecen a un **único node group**, tienda-nodes, asociado a las 4 subredes privadas, con deseado 2 / mínimo 2 / máximo 4. No es un node group por zona: es uno solo y su Auto Scaling Group reparte los nodos entre las 2 AZ, lo que da alta disponibilidad. El **Cluster Autoscaler** —que ajustaría el número de nodos solo— queda documentado como mejora futura.”

8 · Flujo de la petición

Señalo: la línea verde del usuario bajando por el namespace.

“Ahora el camino real de una petición, que baja en línea recta: el **usuario** entra por el ELB en el puerto 80; eso llega al Service tienda-frontend (tipo LoadBalancer) que reparte a sus 2 pods Nginx; el frontend llama al backend por /api en el puerto 3001 (Service ClusterIP); y el backend consulta MySQL en el puerto 3306 vía su service headless.”

9 · Autoscaling (HPA) — lo central

Señalo: las cajas amarillas de HPA y el Metrics Server a la derecha.

“Esto es el corazón del encargo. El pipeline verifica la Metrics API (v1beta1.metrics.k8s.io) y aplica los HPA. El **Metrics Server** lee la CPU de los pods y el HPA agrega o quita réplicas: el frontend con umbral 60% escala de 2 a 6, y el backend con umbral 70% escala de 2 a 10. Esto funciona porque cada deployment declara **requests/limits de CPU** (el backend 100m/500m).”

“En la prueba de carga real, el backend subió de 2 hasta 10 pods con la CPU sobre 400%, y luego bajó solo a 2. Y aquí cierro el concepto: el **HPA escala pods, no nodos**; esos pods nuevos nacen sobre los mismos 2 nodos. Escalar nodos sería tarea del Cluster Autoscaler.”

10 · Secret de credenciales

Señalo: la caja roja mysql-secret a la izquierda del namespace.

“Este Secret no está en el repo: lo **genera el pipeline** desde los secretos de GitHub (paso que mostré antes), y de aquí lo consumen el backend —vía secretKeyRef en DB_PASSWORD— y la base de datos. Es la evidencia directa de IE5.”

Cierre

Señalo: la leyenda de indicadores al pie.

“Esta leyenda conecta cada parte con los 7 indicadores: infraestructura EKS, ELB y URL pública, autoescalado con HPA, pipeline CI/CD, secrets/Environment, logs y autorrecuperación.”

Frase de cierre: un git push entra por arriba, el pipeline hace build y push a ECR, despliega con kubectl sobre nodos privados y, sin intervención manual, la app queda corriendo de forma segura, con alta disponibilidad y autoescalado, llegando al usuario por el ELB.

Anexo A — Desglose del pipeline `.github/workflows/deploy-eks.yml`

Disparadores: `on: push (branches: [main])` y `workflow_dispatch`. Job único `build-and-deploy` sobre `runs-on: ubuntu-latest` con `environment: production` (expone los 7 secrets). Variables `env`: `ECR_REPO_FRONTEND/BACKEND/DB`.

#	Step (name del YAML) · action	Qué hace
1	Checkout código · <code>actions/checkout@v4</code>	Clona el repo en el runner.
2	Configurar credenciales AWS · <code>configure-aws-credentials@v4</code>	Carga <code>AWS_ACCESS_KEY_ID</code> , <code>SECRET</code> , <code>REGION</code> y <code>SESSION_TOKEN</code> (token obligatorio: credenciales temporales del Learner Lab).
3	Login a Amazon ECR · <code>amazon-ecr-login@v2</code>	Autentica Docker contra ECR; expone <code>steps.login-ecr.outputs.registry</code> .
4	Definir tag de imagen	<code>IMAGE_TAG=\${GITHUB_SHA:0:7}</code> → tag trazable al commit.
5-7	Build & push FRONTEND / BACKEND / DB	Por cada uno: <code>docker build</code> con tags <code>:<sha7></code> y <code>:latest</code> , luego <code>docker push</code> ambos a su repo de ECR.
8	Instalar kubectl · <code>azure/setup-kubectl@v4</code>	Instala <code>kubectl v1.29.0</code> en el runner.
9	Configurar kubeconfig para EKS	<code>aws eks update-kubeconfig --region \$AWS_REGION --name \$EKS_CLUSTER_NAME + kubectl config current-context</code> .
10	Aplicar namespace	<code>kubectl apply -f k8s/namespace.yaml</code> (crea/asegura el ns tienda).
11	Crear/actualizar Secret de MySQL	<code>kubectl create secret generic mysql-secret --from-literal=MYSQL_ROOT_PASSWORD=... -n tienda --dry-run=client -o yaml kubectl apply -f -</code> . La contraseña nunca se versiona.
12	Desplegar BASE DE DATOS (MySQL)	<code>apply deployment+service, set image deployment/tienda-db mysql=...:<sha7>, rollout status</code> .
13	Desplegar BACKEND	<code>apply deployment+service, set image deployment/tienda-backend backend=..., rollout status</code> .
14	Desplegar FRONTEND	<code>apply deployment+service, set image deployment/tienda-frontend frontend=..., rollout status</code> .
15	Verificar Metrics Server (para HPA)	<code>kubectl get apiservices v1beta1.metrics.k8s.io + kubectl wait --for=condition=Available</code> . No instala el manifiesto upstream (usa el add-on gestionado).
16	Aplicar HPA	<code>kubectl apply -f k8s/backend-hpa.yaml y frontend-hpa.yaml</code> (si existen).
17	Ver pods, servicios y HPA	<code>kubectl get pods -o wide, get svc, get hpa</code> (evidencia en los logs).

Anexo B — Desglose de los manifiestos `k8s/*.yaml`

Todos llevan `namespace: tienda`. El pipeline aplica `deployment + service` y luego sobrescribe la imagen con `kubectl set image`, por eso el tag `:latest` del YAML es solo un valor inicial.

Archivo	Kind	Campos clave
<code>namespace.yaml</code>	Namespace	Crea el namespace <code>tienda</code> que aísla todos los recursos.
<code>mysql-secret.example.yaml</code>	Secret (plantilla)	Solo referencia: el Secret real <code>mysql-secret</code> lo genera el pipeline (step 11). Evita versionar la clave.
<code>mysql-deployment.yaml</code>	Deployment	<code>replicas: 1</code> · contenedor <code>mysql</code> puerto <code>3306</code> · <code>MYSQL_ROOT_PASSWORD</code> vía <code>secretKeyRef</code> · volumen <code>emptyDir</code> en <code>/var/lib/mysql</code> .
<code>mysql-service.yaml</code>	Service	<code>clusterIP: None (headless)</code> · puerto <code>3306</code> · DNS interno <code>tienda-db</code> .
<code>backend-deployment.yaml</code>	Deployment	<code>replicas: 2</code> · puerto <code>3001</code> · env <code>DB_HOST=tienda-db</code> , <code>DB_USER=root</code> , <code>DB_PASSWORD</code> (<code>secretKeyRef</code>), <code>DB_NAME=tienda_perritos</code> , <code>DB_PORT=3306</code> · resources req <code>100m/128Mi</code> · lim <code>500m/512Mi</code> · probes <code>readiness/liveness /api/health</code> .
<code>backend-service.yaml</code>	Service	<code>type: ClusterIP</code> · puerto <code>3001</code> (solo interno).
<code>backend-hpa.yaml</code>	HorizontalPodAutoscaler	<code>autoscaling/v2</code> · target <code>tienda-backend</code> · <code>minReplicas: 2</code> · <code>maxReplicas: 10</code> · CPU Utilization <code>70%</code> .
<code>frontend-deployment.yaml</code>	Deployment	<code>replicas: 2</code> · puerto <code>80</code> · resources req <code>50m/64Mi</code> · lim <code>300m/256Mi</code> · probes <code>readiness/liveness /</code> .
<code>frontend-service.yaml</code>	Service	<code>type: LoadBalancer</code> · puerto <code>80</code> → EKS aprovisiona el ELB Classic público (sin AWS LB Controller en el Learner Lab).
<code>frontend-hpa.yaml</code>	HorizontalPodAutoscaler	<code>autoscaling/v2</code> · target <code>tienda-frontend</code> · <code>minReplicas: 2</code> · <code>maxReplicas: 6</code> · CPU Utilization <code>60%</code> .

Relación con el HPA: los **requests de CPU** son imprescindibles — el HPA calcula el % de uso sobre el request, no sobre el límite. Sin resources no habría métrica y el HPA quedaría en `<unknown>`.

Anexo C — Los 7 secrets del Environment production

Viven en GitHub → Settings > Environments > production (no en el repositorio). El job solo puede leerlos porque declara `environment: production`, y los referencia como `${{ secrets.NOMBRE }}`. Las tres credenciales de AWS son **temporales y rotan en cada sesión** del Learner Lab, por eso hay que actualizarlas al empezar a trabajar.

Secret	Qué es	De dónde se extrae	Dónde lo usa el pipeline
AWS_ACCESS_KEY_ID	Identificador público de la credencial temporal de AWS.	AWS Academy Learner Lab → AWS Details → AWS CLI: Show (bloque de credenciales). Rota por sesión.	Step 2 <code>configure-aws-credentials</code> .
AWS_SECRET_ACCESS_KEY	Clave secreta asociada (la 'contraseña' de esa credencial).	Mismo bloque del Learner Lab (AWS Details > AWS CLI).	Step 2 <code>configure-aws-credentials</code> .
AWS_SESSION_TOKEN	Token de sesión temporal (STS), obligatorio para credenciales temporales.	Mismo bloque del Learner Lab; cambia cada vez que se reinicia el lab.	Step 2 <code>configure-aws-credentials</code> .
AWS_REGION	Región de AWS donde vive toda la infraestructura.	Valor fijo definido por el equipo: <code>us-east-1</code> .	Steps 2 y 9 (<code>configure-aws-credentials</code> y <code>update-kubeconfig</code>).
EKS_CLUSTER_NAME	Nombre del clúster EKS al que conectarse.	Definido al crear el clúster en la consola: <code>tienda-eks</code> .	Step 9 <code>aws eks update-kubeconfig --name ...</code>
EKS_NAMESPACE	Namespace de Kubernetes destino del despliegue.	Definido en <code>k8s/namespace.yaml</code> : <code>tienda</code> .	Steps 11–17 (todos los <code>kubectl ... -n \$EKS_NAMESPACE</code>).
MYSQL_ROOT_PASSWORD	Contraseña root de la base de datos MySQL.	Valor definido por el equipo (no se publica en el informe ni en el repo).	Step 11: genera el Secret <code>mysql-secret</code> , que backend y db consumen vía <code>secretKeyRef</code> .

Por qué importa (IE5): los secrets nunca se escriben en el código. Las credenciales de AWS habilitan el push a ECR y el acceso a EKS; la `MYSQL_ROOT_PASSWORD` no se versiona — el pipeline la convierte en un Secret de Kubernetes **en tiempo de despliegue**, y de ahí la leen los pods. Si algún valor expira (sesión del lab vencida), el pipeline falla en el step 2 y basta con actualizar los tres secrets de AWS.

Anexo D — Para qué sirve cada manifiesto (Deployment, Service, HPA)

Kubernetes separa responsabilidades: un objeto define **el ciclo de vida de los pods**, otro **cómo se les llega por red**, y otro **cuántos pods hay según la carga**. Por eso hay tres tipos distintos en lugar de uno solo.

Tipo	Para qué sirve	En tu proyecto
Namespace	Crea un espacio lógico que aísla y agrupa todos los recursos.	tienda contiene front, back, db, services, HPA y secret.
Deployment	Define el estado deseado de la app: qué imagen, cuántas réplicas, recursos y probes. Crea y mantiene los pods, los recrea si mueren (autorrecuperación) y gestiona los rollouts.	3 deployments: tienda-frontend (2), tienda-backend (2), tienda-db (1).
Service	Da una dirección de red estable (IP/DNS fijo) y balancea el tráfico entre los pods sanos. Necesario porque las IP de los pods cambian al recrearse.	Uno por componente; tres tipos según cómo se accede (abajo).
• ClusterIP	Service solo accesible dentro del clúster.	tienda-backend:3001 (lo llama el frontend).
• LoadBalancer	Expone el Service hacia afuera ; en EKS aprovisiona un ELB público.	tienda-frontend:80 → ELB Classic público.
• headless	Sin IP virtual (clusterIP: None); DNS directo al pod.	tienda-db:3306 (acceso interno a MySQL).
HorizontalPodAutoscaler	Ajusta automáticamente el número de réplicas del Deployment según el uso de CPU (leído del Metrics Server).	backend 2→10 @70%, frontend 2→6 @60%. La db no tiene HPA (es stateful).
Secret	Guarda credenciales fuera del código; los pods las leen vía secretKeyRef.	mysql-secret, generado por el pipeline (Anexo C).

Por qué tres objetos y no uno: es **separación de responsabilidades**. El Deployment se ocupa de **qué corre y cuántas copias base**; el Service de **cómo se llega** a esas copias; y el HPA de **cuántas copias según la carga**. Así cada uno cambia sin romper a los otros.

¿HPA y escalado de nodos son redundantes? **No**. El HPA escala **pods** (copias de la app); el escalado del node group escala **nodos** (máquinas EC2). Son capas distintas y complementarias: si el HPA pide más pods de los que caben, quedarían en Pending hasta que haya un nodo libre —y agregar nodos automáticamente sería tarea del Cluster Autoscaler (mejora futura). Hoy el HPA es dinámico y el número de nodos es fijo (2-4).

Anexo E — Qué defines en ECR y qué defines en EKS

Son dos servicios con roles distintos: **ECR** (Elastic Container Registry) es **dónde viven las imágenes**; **EKS** (Elastic Kubernetes Service) es **dónde y cómo corre la aplicación**. ECR solo almacena; EKS ejecuta (y hace pull de ECR).

Servicio / capa	Qué defines ahí	En tu proyecto
ECR (registro de imágenes)	Los repositorios y su configuración: privado/público, mutabilidad de tags, cifrado y región. No defines cómo corre la app: solo guarda imágenes versionadas.	3 repos tienda-frontend/backend/db, privados, Mutable, AES-256, us-east-1. Tags :<sha7> y :latest.
EKS · clúster	La base del orquestador : versión de Kubernetes, endpoint (público/privado), add-ons, rol IAM y cifrado.	tienda-eks, K8s 1.35, endpoint público y privado, add-ons CoreDN S/kube-proxy/VPC CNI/Metrics Ser ver, rol LabRole.
EKS · node group (capa de cómputo)	Las máquinas (nodos) donde corren los pods: tipo de instancia, AMI, disco, rango de escalado, subredes y acceso SSH.	tienda-nodes, t3.medium, AL2023, disco 20 GB, escalado 2/2/4, 4 subredes privadas, SSH OFF.
EKS · workloads (manifiestos k8s/)	Cómo corre la app : namespace, deployments (imagen, réplicas, recursos, probes), services, HPA y secret. Es lo que aplica el pipeline con kubectl.	ns tienda, 3 deployments, 3 services, 2 HPA, mysql-secret.

Regla simple: si es una **imagen** (qué empaqueto y subo) → va en ECR. Si es **infraestructura o ejecución** (qué clúster, cuántos nodos, cómo corren y escalan los pods) → va en EKS.

Nota — ECS no es lo mismo que EKS: ambos son orquestadores de contenedores de AWS, pero ECS es el propio de AWS (sin Kubernetes) y EKS es Kubernetes gestionado. En este proyecto **se eligió EKS**; ECS no se utiliza. Lo que sí acompaña a EKS es ECR (el registro de imágenes).

Anexo F — Cómo se relaciona el clúster EKS con el Classic ELB

El ELB es el único componente que **vive fuera del clúster pero lo crea y administra el propio clúster**. La relación tiene dos caras: cómo nace y cómo lleva el tráfico hasta los pods.

F.1 — El clúster crea el ELB (no se configura a mano)

Todo arranca por la línea `type: LoadBalancer` del `frontend-service.yaml`.

- 1 El `kubectl apply` del Service llega al **API server** del plano de control.
- 2 Dentro del plano de control corre el **cloud-controller-manager** (la integración de Kubernetes con AWS), que vigila los Services.
- 3 Al ver uno de `type: LoadBalancer`, llama a la **API de AWS** para aprovisionar un balanceador.
- 4 Como en el Learner Lab **no está el AWS Load Balancer Controller**, AWS crea un **Classic ELB** público en las **subredes públicas**, escuchando en el `:80`, con su DNS `...elb.amazonaws.com`.
- 5 Ciclo de vida: el ELB queda **atado al Service**. Si se borrara el Service, EKS **elimina el ELB** automáticamente.

F.2 — Cómo el ELB hace llegar el tráfico al pod

Clave: el ELB **no apunta a los pods** (sus IP cambian), apunta a los **nodos**. La cadena completa es:

Usuario `:80` → **Classic ELB** (DNS público, subredes públicas) → **Nodo EC2** por un **NodePort** (puerto que el Service abre en TODOS los nodos) → **kube-proxy** (reglas iptables en el nodo) → **Pod frontend** (Nginx `:80`)

- Al crear el Service `LoadBalancer`, Kubernetes le asigna por debajo un **NodePort**: un puerto alto abierto en **todos** los nodos.
- El ELB **registra los nodos como destinos** (targets) y les hace **health checks**; reparte las conexiones entre ellos.
- `kube-proxy` (add-on) en cada nodo toma lo que entra por ese `NodePort` y lo **reenvía a un pod del frontend, aunque esté en otro nodo**. Por eso no importa en qué nodo cayó cada pod.
- Requisito de red: el ELB necesita una **subred pública por AZ** — por eso el diseño tiene 2 subredes públicas.

Frase de defensa: “No configuré el ELB a mano: declaré el Service del frontend como `type: LoadBalancer` y el plano de control de EKS aprovisionó un Classic ELB público. Balancea hacia los nodos por un `NodePort`, y `kube-proxy` reenvía al pod de Nginx; si borrara el Service, EKS eliminaría el ELB.”

Anexo G — Add-ons del clúster (cuáles hay y por qué son necesarios)

En el wizard de creación se marcaron **4 add-ons gestionados**. Son piezas que Kubernetes necesita para funcionar (red, DNS, métricas); AWS las instala, parcha y actualiza por nosotros.

Add-on	Qué hace	Por qué es necesario aquí
Amazon VPC CNI	Plugin de red: asigna a cada pod una IP real de la VPC (tomada de las subredes).	Sin red un pod no existe. Integra los pods a la VPC y permite que kube-proxy y el ELB los alcancen.
kube-proxy	Programa las reglas (iptables) en cada nodo que enrutan el tráfico de un Service hacia sus pods.	Sin él los Services no funcionan: es lo que hace el NodePort del LoadBalancer y el ClusterIP del backend.
CoreDNS	DNS interno del clúster: resuelve nombres de Service a IP .	El backend encuentra a tienda-db por su nombre; sin DNS no hay descubrimiento de servicios entre pods.
Metrics Server	Recolecta CPU y memoria de los pods y los expone en la Metrics API (v1beta1.metrics.k8s.io).	Imprescindible para el HPA (sin él queda en <unknown>) y para <code>kubectl top</code> .

Los tres primeros (VPC CNI, kube-proxy, CoreDNS) son el **núcleo de red** que todo clúster EKS necesita; el Metrics Server se añadió expresamente para que el HPA tuviera métricas de CPU (resuelve el problema clásico del HPA en <unknown> sin instalar el metrics-server de upstream a mano).

Add-ons que se DESMARCARON a propósito (no necesarios y para ahorrar créditos): **Node monitoring agent** (observabilidad extra), **EKS Pod Identity Agent** (no se usa: los permisos van por el rol `LabRole` del nodo), **External DNS** (no hay dominio Route 53) y todos los de terceros.

Anexo H — Los flujos de la arquitectura (resumen)

La arquitectura tiene **2 flujos** (despliegue y petición del usuario) y **1 propiedad** transversal (seguridad de red). El 'interactuar con el frontend' y el 'entrar por el DNS del ELB' son **el mismo flujo**, no dos.

Flujo 1 — Despliegue (lo dispara un commit)

- No es un 'reset': es un **rolling update** (redespliegue gradual).
- Un push a main reconstruye las 3 imágenes y redespliega los 3 deployments (no solo el que cambió, por el tag con SHA nuevo).
- No los mata de golpe: salen pods nuevos y recién entonces mueren los viejos — sin caída, con 2 réplicas.
- Efecto colateral: el pod de MySQL también se reemplaza y, por emptyDir, la base se reinicia vacía.

Flujo 2 — Petición del usuario (frontend y ELB son el mismo salto)

```
Usuario → DNS del ELB :80 → Service frontend → pods Nginx → Service backend :3001 /api → pods Node/Express → Service db :3306 → MySQL
```

Por qué el ELB está asociado al frontend: porque **solo el Service del frontend es** type: LoadBalancer. EKS le crea un ELB público y su selector apunta a los pods app: tienda-frontend. El backend (clusterIP) y la db (headless) son **solo internos**, sin ELB — a propósito: el frontend es la cara pública, el resto no debe verse desde Internet.

Propiedad transversal — Seguridad de red (sin SG personalizados)

Corrección importante: no es que 'no haya grupos de seguridad'. Los hay y protegen, pero los **gestiona EKS/AWS automáticamente**, no hubo que crear ninguno personalizado:

- EKS crea un **security group del clúster** que permite la comunicación control plane ↔ nodos y nodo ↔ nodo.
- El ELB obtiene su **propio security group** automáticamente (abre el :80 y las reglas hacia el NodePort de los nodos).

La seguridad real viene del **diseño de red** que sí se decidió:

- Nodos en **subredes privadas, sin IP pública** → inalcanzables desde Internet.
- **NAT** → los nodos pueden salir (pull de ECR) pero **nadie entra**.
- Exposición pública **limitada al frontend**; backend y db quedan internos.
- Endpoint del clúster público y privado → el tráfico nodo↔API se queda dentro de la VPC.

Frase de defensa: “No creé grupos de seguridad personalizados porque EKS aprovisiona y gestiona automáticamente los del clúster y del ELB. La seguridad viene del diseño de red: nodos en subredes privadas sin IP pública, salida solo por NAT, y exposición pública limitada al frontend con un único Service LoadBalancer; backend y base de datos permanecen internos.”